An Observation of the Industry Practice of MongoDB

Introduction

This essay introduces the methodology used in the student's project. First, it explains the reason for selecting the methodology. Second, a literature review is presented as well as the advantages and disadvantages of the methodology. Next, an observation of the industry practice for the methodology is explained. Lastly, the comparison between the theory and the observation is discussed.

Background

MongoDB NoSQL database is the methodology chosen for this project. It is one of the most popular document-oriented databases that help companies build scalable applications.

MongoDB is the database used by SchoolBell.chat. Therefore, end users' information is stored in the database that supports creating, reading, updating and deleting data. These functionalities will be specifically used in the designing and coding phases of the project.

MongoDB is chosen because it allows the student to observe how it will be used in actual work, and to gain hands-on experience as well as an understanding of the NoSQL database.

Growing Data

With the increasing amount of data, how to handle non-uniform data has become a challenge (Gupta et al., 2017). This problem creates an opportunity for NoSQL (Not only SQL) databases because of their scalability, efficiency and ability to handle a huge volume of data (Gupta et al., 2017).

NoSQL

NoSQL databases are non-relational databases. Table, row and column are not applied for data storage and retrieval. NoSQL databases provide horizontal scalability, and they handle an increased number of servers rather than hardware upgrade (Gupta et al., 2017).

Generally, four types of NoSQL databases are identified, including (Gupta et al., 2017; Parmar & Roy, 2018; Sahatqija et al., 2017)

- Graph database
- Key-Value store
- Column store
- Document database

Particularly, the Document database is considered as the most popular type of NoSQL database (Parmar & Roy, 2018). It can handle structured, semi-structured and unstructured data (Gupta et al., 2017). Also, it is like the key-value store, but values in a document database are saved as documents which are data with complex structures (Gupta et al., 2017).

NoSQL database system saves multiple documents as a collection, and a document can include multiple pairs of key-value, key-array or can include a nested document (Gupta et al., 2017).

As NoSQL databases are horizontal scalable and they do not need to follow schema method in Relational Database Management Systems (RDBMSs), they are easy to manage and can efficiently handle huge amounts of data (Gupta et al., 2017).

Compared with the instant data consistency, powerful query language and structured data storage emphasised by RDBMS, many NoSQL databases are distributed data storage that concentrates on high performance, high availability, data replication and scalability (Gupta et al., 2017).

MongoDB

Among document-based databases, MongoDB is one of the most popular database management systems (Gupta et al., 2017). It is developed in C++ and has been used as back-end software since 2009 (Agrawal et al., 2015)

Also, MongoDB is an open-source document-oriented database (Agrawal et al., 2015; MongoDB, 2020; Parmar & Roy, 2018) built on a horizontal scale-out architecture (MongoDB, 2020). This architecture allows it to handle a large amount of data and traffic (MongoDB, 2020).

MongoDB is a schema-less database, and it allows implementation in a distributed environment to provide flexibility (Deari et al., 2018). Collection and document are two important concepts of MongoDB:

- **Collection**. It is stored in the MongoDB database like a table in a relational database management system. It is a group of documents (Agrawal et al., 2015; Parmar & Roy, 2018).
- **Document**. It consists of multiple key-value pairs. Data is stored in the form of JSON and BSON (MongoDB, 2020). Documents have a dynamic schema that can store different types of data in the same collection (Agrawal et al., 2015; Parmar & Roy, 2018).

According to MongoDB (2020), MongoDB has been used to support applications in various domains. For example, IoT, Gaming, Logistics and e-Commerce. More specifically, Expedia uses MongoDB as a back-end to automate process of notetaking and store all the data (Agrawal et al., 2015).

Organisations use MongoDB because it provides the developers with a powerful way to handle different types of data in a scalable approach. As a result, they can move to the next step quickly (MongoDB, 2020).

Critique (Pros and Cons)

Advantages of MongoDB includes:

- Scalability
- Sharding
- Dynamic Schema
- Speed

Scalability

MongoDB is horizontally scalable. When handling the growing huge amount of data, MongoDB can scale easily and efficiently (Gupta et al., 2017; **Parmar & Roy, 2018**).

Sharding

MongoDB uses sharding to handle a vast amount of data by spreading data across different servers that connect to the application (Agrawal et al., 2015).

Dynamic Schema

Without explicitly pre-define the schema of the database, developers can flexibly start their work rather than waiting for schema designing (Parmar & Roy, 2018).

Speed

Modelling data in a document-oriented database will maximise the speed by using indexes to quickly get data from the database (Agrawal et al., 2015).

Disadvantages of MongoDB includes:

- Data Redundancy
- No real use of JOIN
- Using high memory for data storage

Data Redundancy

MongoDB stores data in collections and uses embedded documents, which can encourage data redundancy (Deari et al., 2018; Gupta et al., 2017; Sahatqija et al., 2017).

No real use of JOIN

MongoDB does not support the real implementation of joins like relational databases. Therefore, it uses nested documents, and it results in data redundancy (Agrawal et al., 2015; Deari et al., 2018).

Using high memory for data storage

MongoDB uses more memory to store key-value pairs, and the nested documents will increase the unnecessary usage of memory (Agrawal et al., 2015).

Setting Up MongoDB on Local Machine for Industry Practice

When the project started, MongoDB was required to install from a link provided by the onboarding training: https://docs.mongodb.com/manual/installation/. There were two editions available to install, and the community edition was selected according to a reply from the industry.

Firstly, the student installed of Xcode command-line tool and MongoDB Homebrew Tap according to official instruction because the student used Mac operating system.

Once they were installed, Compass, the GUI for MongoDB was downloaded and installed from the link: https://docs.mongodb.com/compass/current/install/. This tool helps the student to explore the data visually.

When MongoDB and Compass were ready, the student followed the instruction provided in the link: https://docs.mongodb.com/compass/current/connect/ to connect the Compass with MongoDB.

Lastly, the student copied the connection string provided by the industry, pasted it into the box, clicked the "Connect" button and the connection was successful.

Overall, the installing links were posted within the onboarding training section on MS Teams.

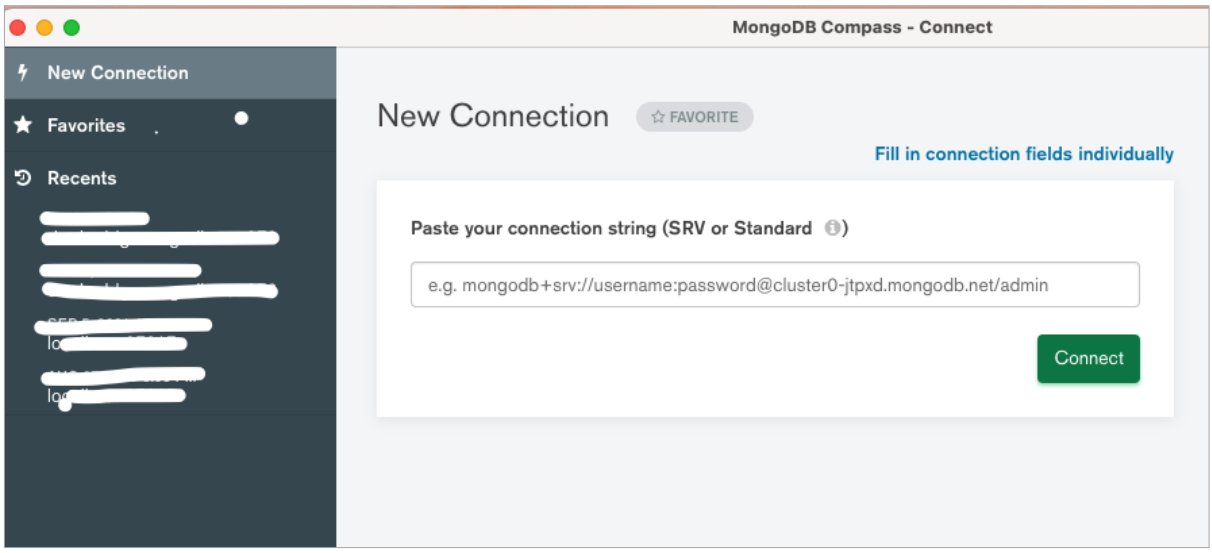

Figure 1 Used connection string

Use MongoDB in Practice

When doing the tasks of creating, viewing and deleting a token, the student created a collection called "oneTimeIntegrationToken" within one of their often-used databases.

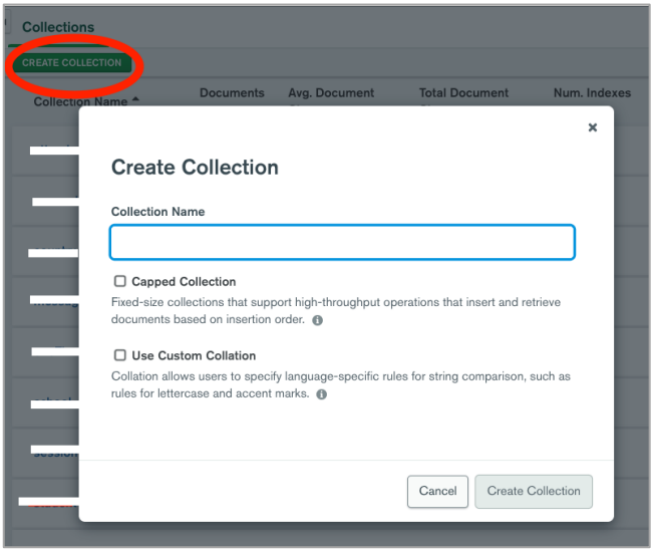

Figure 2 Create a collection

Creating a collection via Compass was straightforward:

1. Click on the "Create Collection" button
2. Enter the collection name
3. Click "Create Collection"

Once done, the collection was displayed as follow:

Figure 3 The collection of oneTimeIntegrationToken

Documents could be added through Compass by clicking and entering information. But the following sections will focus on the industry practice of modifying documents by coding.

Model Code

A model that has the exact same name as the created collection was developed in the codebase. It contained the properties that will be used as keys in the database. The following model code was written by the student and reviewed and updated by the team.

```
export interface OneTimeIntegrationToken
{
   _id?: string;
   issuedTime: Date;
   token: string;
   schoolId: string;
   administrator: xxxxx;
   isActive: boolean;
}
```

Token Service

To connect the back-end service with MongoDB, environment variables of the database were set in the serverless.yml of the service (sample code by other professionals).

```
environment:
   MONGO_DB_URL: xxxxxxxxxxxxxxxxx
   MONGO_DB_DB_NAME: xxxxxxxxxxxxx
```

The environment variables could be checked through Amazon Web Services (AWS) Lambda. It just needs to search the function name and then direct to the “Configuration” tab. The student came here to check the database name when errors were related to the database.

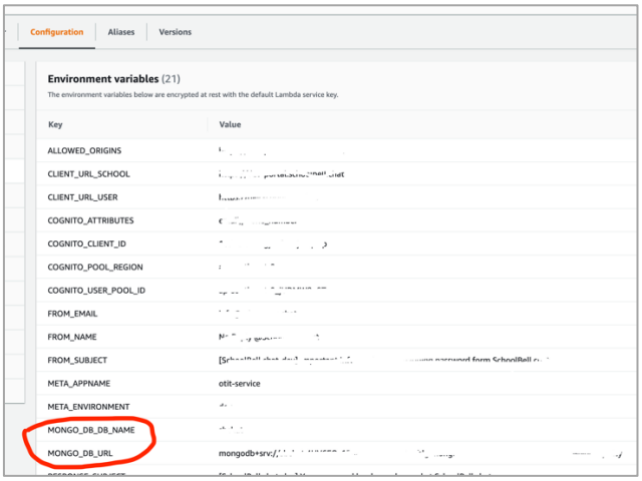

Figure 4 AWS Environment Variables - MongoDB

Within the function of createToken (), the token was saved into the database via the following method (sample code by teamwork). Then, the function saveToken () called an API service.

```
this.onetimeTokenDBService.saveToken()
```

API Database Service

Apart from the token service, an API database service was created for the creating, getting and deleting the token. In the function of creating a token, the following code was used to write data (token) into the collection named "oneTimeIntegrationToken" (sample code by the student):

```
this.write(token, `oneTimeIntegrationToken`)
```

The above line of code would trigger the write (token, collection) method in the database connection file to connect to the MongoDB (sample code by other professionals):

```
MongoClient.connect(this.url, { useNewUrlParser: true, useUnifiedTopology: true })
…
await client.db(this.dbName).collection(collection).insertOne(message)
```

According to MongoDB (2021a), "db.collection.insertOne()" is used to insert a document data into a collection of a database. Eventually, the insertOne () function inserted an object (in the example, it called OneTimeIntegrationToken) into the collection "OneTimeIntegrationToken" in the database.

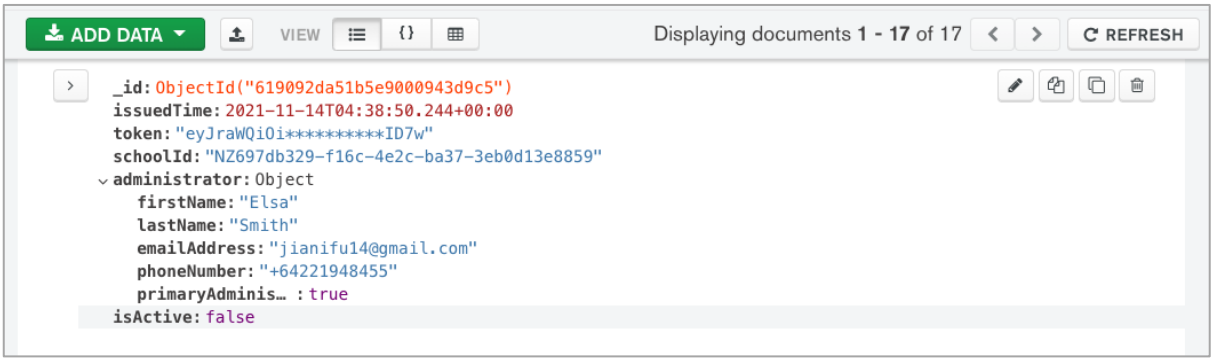

Figure 5 Successful insert data into a collection of the database – Compass GUI

Similarly, when getting/finding an existing token in the database, the API database service called (sample by the student):

```
this.findOne({ schoolId: schoolId}, `oneTimeIntegrationToken`)
```

By checking the AWS Logs of getToken () method, a JSON object was returned as shown in Figure 6.


```
() Query: {"schoolId":"NZ697db329-f16c-4e2c-ba37-3eb0d13e8859"}
asswords will not be sanitized
() Value returned [object Object]
"_id":"61857b6ea93ab10009fd5fab","issuedTime":"2021-11-05T18:43:58.567Z","token":"eyJraWQiOi**********uZUQ","schoolId":"NZ697db329-f16c-4e2c-ba37-3eb0d13e8859","administrator":
eturned: {
```


Figure 6 AWS Log - Getting token

Because of the length of the log, a formatted and complete version is attached below.


```
{
    "_id":"61857b6ea93ab10009fd5fab",
    "issuedTime": "2021-11-05T18:43:58.567Z",
    "token": "eyJraWQiOi**********uZUQ",
    "schoolId": "NZ697db329-f16c-4e2c-ba373eb0d13e8859",
    "administrator":
    {
        "firstName": "Elsa",
        "lastName": "Smith",
        "emailAddress": "jianifu14@gmail.com",
        "phoneNumber": "+64221948455",
        "primaryAdministrator": true
    },
    "isActive": true
}
```


Because the business logic only required returning one valid token for each school, the student decided to use findOne () instead of find (). The findOne () method will return a document that fulfils the query. When the method finds multiple documents that fulfil the query, the first document that was inserted early will be returned (MongoDB, 2021).

To achieve soft deletion of an existing token in the database, the updateOne () method was called (sample code by other professionals):

```
this.updateOne(`oneTimeIntegrationToken`, { schoolId: schoolId }, {$set: {isActive: false}} )
```

The above line of code told the database to find the school by school ID in the collection named "oneTimeIntegrationToken" and set the key (isActive) to "false", rather than remove the data from the collection.

In addition to the token tasks, the student also worked on tasks of inserting/updating/deleting student information. The query tried to replace the relevant fields in "FETCH_RELATIONSHIP" with the method's parameters (sample code by the student), and fetch the matched student information.

```
const qry = JSON.stringify(FETCH_RELATIONSHIP)
        .replace('SCHOOL_ID', schoolId)
        .replace('STUDENT_ID', studentId)
        .replace('GROUP', group)
        .replace('PARENT_EMAIL', parentEmailAddress)
        .replace('TEACHER_EMAIL', teacherEmailAddress)
        .replace('"IS_ACTIVE"', `${isActive}`);
      this.aggregate(JSON.parse(qry), 'student')
```

The following sample code was produced by other professionals.

```
export const FETCH_RELATIONSHIP = [
...
...
, {
    '$match': {
      'studentId': 'STUDENT_ID',
      'group': 'GROUP',
      'parentEmailAddress': 'PARENT_EMAIL',
      'teacherEmailAddress': 'TEACHER_EMAIL',
      'isActive': 'IS_ACTIVE'
    }
  }
 ]
```

Through observation, the student also found that most of the API services come with a query file in the code base, and they were basically ready to use or just need some update with the methods.

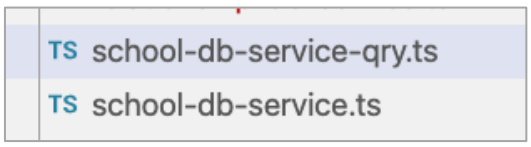

Figure 7 Example of a service comes with query file

Compass

During the development of the token feature, the student tested the functionality by creating/deleting token on the web page multiple times. It was inconvenient but Compass helped the student to address the issue. The GUI tool provides buttons for users to manage the data in a friendly way. As shown in Figure 5, four buttons sit on the top-right corner. The first one on the right-hand side was used by the student to quickly delete an existing token, then she did not need to follow the whole deleting steps from the web page. Also, when the function of soft deletion was available, the student often modified the value of the key (isActive) from true to false. In this way, the token could be considered as deleted from the perspective of end-users.

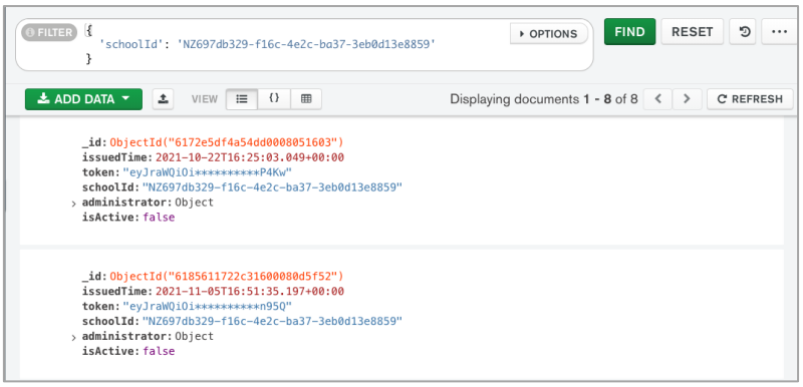

Figure 8 Filter - Compass GUI

Apart from the modifying buttons, Compass also provide a searching space for users to filter the data. As shown in Figure 5, the student normally entered a filter document in the searching area to check the history of token.

```
{
        'schoolId': 'NZ697db329-f16c-4e2c-ba37-3eb0d13e8859'
}
```

In addition to filter, Compass has a simple use interface for aggregations. As shown in Figure 8, by using $count, there were 17 documents related to the provided school ID.

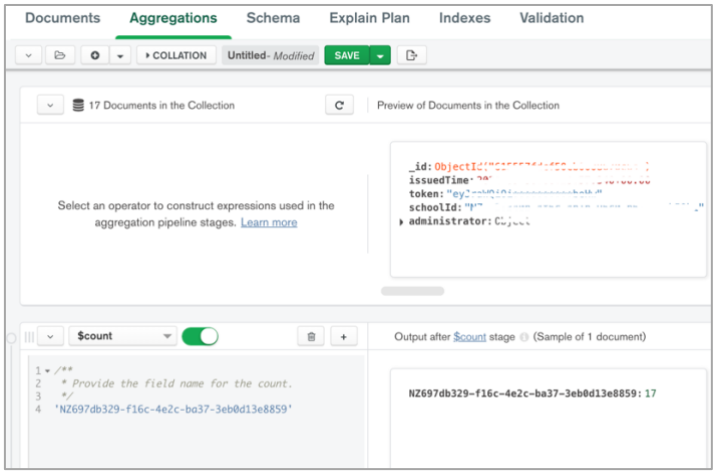

Figure 9 Aggregations

Overall, Compass has a simple and clear user interface that can help users to quickly create collections. Documents can also be generated from the interface. By using the filter box or aggregations, data can be filtered.

Comparing the Observation to the Theory

From the industry observation, indeed, as a NoSQL database, MongoDB does not store data in traditional tables that have columns and rows (Gupta et al., 2017). Also, MongoDB does not require a fixed schema, compared to the relational database management system (Deari et al., 2018). Alternatively, collections and documents were the two keywords of MongoDB during the student's industry practice. One document can nest multiple documents or pairs of key-value.

Also, it was relatively simple to manipulate the data. For example, the school token can be easily set to invalid by using one line of the query below in the previous example:

```
this.updateOne(`oneTimeIntegrationToken`, { schoolId: schoolId }, {$set: {isActive: false}} )
```

Considering the above two characteristics, MongoDB should own advantage to handle a large amount of data, and this matches the perspective of Gupta et al. (2017) and Parmar & Roy (2018).

Moreover, by installing the MongoDB and Compass on the local machine, the student noticed that there was a "Cluster"/ "Replica Set" in the interface of Compass (refer to Figure 10). According to MongoDB (2021b), the replica set is also known as a sharded cluster. It is about flexibility and the data can be spread across multiple servers (Agrawal et al., 2015).

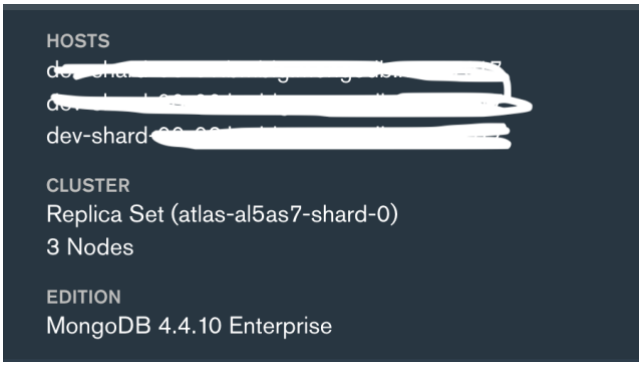

Figure 10 Cluster - MongoDB

Indeed, MongoDB has the advantage of dynamic schema (Parmar & Roy, 2018). When the student was involved in the project, there was less chance to be stuck by database issues. Further, as a developer, the student was able to create a collection and documents directly without waiting for schema design (refer to Figure 2).

However, multiple nested could possibly increase the size of the document as well as decrease the readability. For example, imaging the following document will have 10 administrators:


```
{
        "_id":"61857b6ea93ab10009fd5fab",
        "issuedTime": "2021-11-05T18:43:58.567Z",
        "token": "eyJraWQiOi**********uZUQ",
        "schoolId": "NZ697db329-f16c-4e2c-ba373eb0d13e8859",
        "administrator":
        {
                "firstName": "Elsa",
                "lastName": "Smith",
                "emailAddress": "jianifu14@gmail.com",
                "phoneNumber": "+64221948455",
                "primaryAdministrator": true
        },
        {
                "firstName": "Lisa",
                "lastName": "Smith",
                "emailAddress": "zimoselection@gmail.com",
                "phoneNumber": "+64221948456",
                "primaryAdministrator": true
        },
        "isActive": true
}
```


Also, the student observed that every collection displayed its total and average size in the Compass interface. Therefore, the more documents nested, the more size the collection could have. Data redundancy may be an issue when the data volume increased.

oneTimeIntegrationToken	17	295.5 B	5.0 KB

Figure 11 Collection average size & total size

Conclusion

From the literature review, MongoDB has become one of the most popular database management systems because of the scalability, sharding, dynamic schema and speed. But it also has issues regarding data redundancy, and it would occupy much memory to store data. According to the student's observation, the problem of redundancy could happen when the amount of data become huge. Compared to the redundancy, the student noticed that the multiple nested documents would significantly reduce the readability and a third tool that was used to format the long JSON object would need. Apart from the readability, MongoDB and Compass had provided the student with a nice user experience in terms of the unstructured relationship, less schema and speed.

Reference List

Agrawal, S., Verma, J. P., Mahidhariya, B., Patel, N., & Patel, A. (2015). Survey on Mongodb: an open-source document database. *Database*, *1*(2), 4.

Deari, R., Zenuni, X., Ajdari, J., Ismaili, F., & Raufi, B. (2018, September). Analysis and Comparison of Document-Based Databases With SQL Relational Databases: MongoDB vs MySQL. In *Proceedings of the International Conference on Information Technologies (InfoTech 2018)* (pp. 1-10).

Gupta, A., Tyagi, S., Panwar, N., Sachdeva, S., & Saxena, U. (2017, October). NoSQL databases: Critical analysis and comparison. In *2017 International Conference on Computing and Communication Technologies for Smart Nation (IC3TSN)* (pp. 293-299). IEEE.

MongoDB (2020, September 4). *Why Use MongoDB & When to Use It?*. https://www.mongodb.com/why-use-mongodb

MongoDB (2021a, November 5). MongoDB Manual. https://docs.mongodb.com/manual/

MongoDB (2021b, November 5). MongoDB Cluster. https://www.mongodb.com/basics/clusters

Mukherjee, S. (2019). The Battle between NoSQL Databases and RDBMS. *SSRN Electronic Journal*. https://doi.org/10.2139/ssrn.3393986

Parmar, R. R., & Roy, S. (2018). MongoDB as an Efficient Graph Database: An Application of Document Oriented NOSQL Database. In M. Mittal, V. E. Balas, D. J. Hemanth, & R. Kumar (Eds.), *Data Intensive Computing Applications for Big Data* (pp. 331–358). IOS Press BV. doi: 1.3233/978-1-61499-814-3-331

Sahatqija, K., Ajdari, J., Zenuni, X., Raufi, B., & Ismaili, F. (2018, May). Comparison between relational and NOSQL databases. In *2018 41st international convention on information and communication technology, electronics and microelectronics (MIPRO)* (pp. 0216-0221). IEEE.